



CS 330: HARDWARE AND SYSTEMS FUNDAMENTALS – FALL 2025

Lab 3: Combinational Circuits Design Lab

Due Date: Wednesday, 26th November 2025 at 11:59 PM

Introduction

This lab aims to enhance your practical digital logic circuit design and implementation skills. Throughout this activity, you will explore the fundamental building blocks of combinational circuits and master Logisim Evolution, a tool for logic circuit simulation.

Lab Objectives:

1. Explore Basic Logic Gates and Components in Logisim Evolution

- Understand the functionalities of fundamental logic gates, including AND, OR, and XOR gates, within Logisim Evolution.
- Familiarize yourself with basic components available in Logisim Evolution, such as wires, inputs, and outputs.

2. Design and Simulate a Full Adder

- Implement half adders to grasp the basics of binary addition at the single-bit level in Logisim Evolution.
- Progress to full adders and explore the simulation of multi-bit binary addition using Logisim components.

3. Integrate Components for Complex Circuit Design

- Apply knowledge gained in the previous objectives to integrate various components, including gates, decoders, multiplexers, and adders, into more complex combinational circuits.
- Simulate and analyze the behavior of the integrated circuits to reinforce the understanding of combinational logic and circuit design principles.

Procedure:

In this lab, using the Logisim Evolution simulator, you will construct “complex” combinational circuits in software. You will use these circuits to learn about digital logic.

For this lab, feel free to discuss the problem with another student or a FI if you get stuck at any point. However, you are not allowed to submit another student’s deliverables.

Submit your lab files by **Wednesday, 26th November**. Make sure you answer all the questions and complete the deliverables. (Number them by Part and Step.)

Create a new Logisim Evolution file and name as **ID_ADDER.circ**

Part 1: Designing a Full-Bit Adder

Aim:

This part of the lab activity aims to design a single-bit full-adder and simulate its corresponding circuit behavior. Adders form part of combinational logic circuits, and they are key components of the ALU (Arithmetic Logic Unit) design. ALUs implement basic arithmetic operations such as addition, subtraction, multiplication, and division.

Objectives:

- Understand the principle of a full adder
- Use simulation to verify design

The logical equation of a full adder is given as follows:

$$\text{sum} = a \oplus b \oplus \text{cin}$$

$$\text{cout} = a \cdot b + b \cdot \text{cin} + a \cdot \text{cin}$$

a, *b* and *cin* are inputs (variables) while *sum* and *cout* are outputs (results). The logical equations represent an addition of three single-bit numbers (*a*, *b*, *cin*) giving a result of a double-bit number represented by *sum* (least significant digit) and *cout* (the most significant digit).

A) Draw the truth table for the above logical equations for a full adder.

B) Add a new circuit and call it **fullbitadder**. Design the circuit for a full-bit adder and label it properly.

C) Simulate the design by applying different combinations of inputs and compare the result with the truth tables of the logical equations.

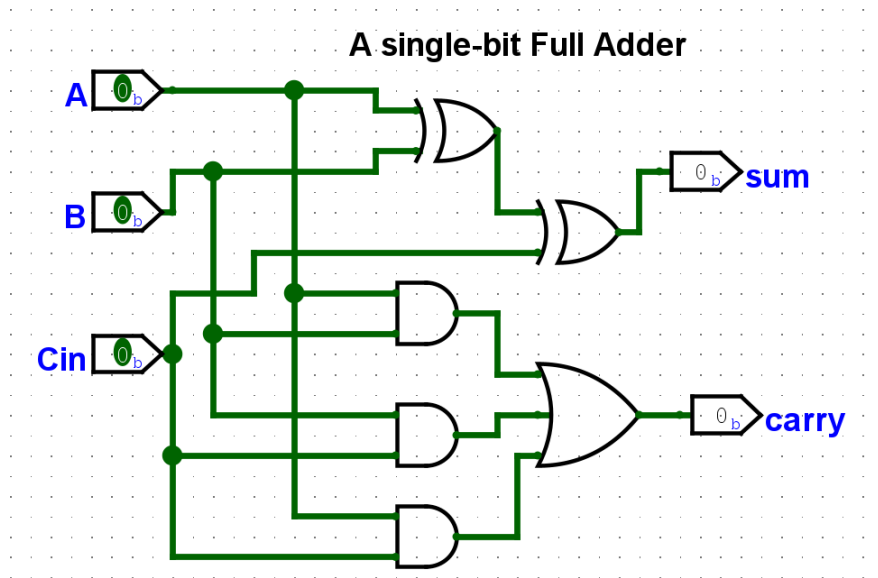


Figure 1: Fulladder circuit in Logisim

Part 2: Designing a 2-bit Full Adder

In the earlier part, you designed a single-bit full adder, meaning the adder can only add 1-bit data inputs. In this part, you will build a 2-bit full adder (a module that will accept two 2-bit numbers and add them) by combining 2 single-bit full adders.

Add a new circuit and name it fulladder2bit. In the same way you create a class while programming to reuse it by creating an object, in digital designing you can create circuits (modules) that can be reused by creating instances of those modules. The symbol of a full adder could look like this:

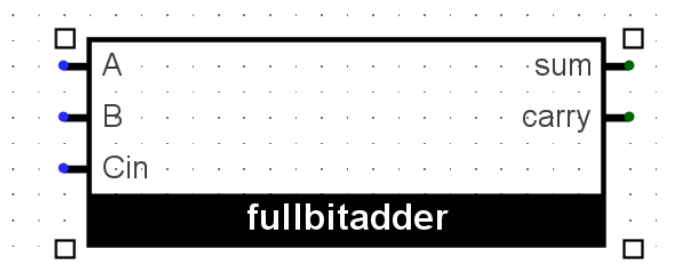


Figure 2: Fulladder mask in Logisim

- Add 2 objects of the *fullbitadder* circuit to the workspace. Interconnect these components to create a 2-bit full-adder.

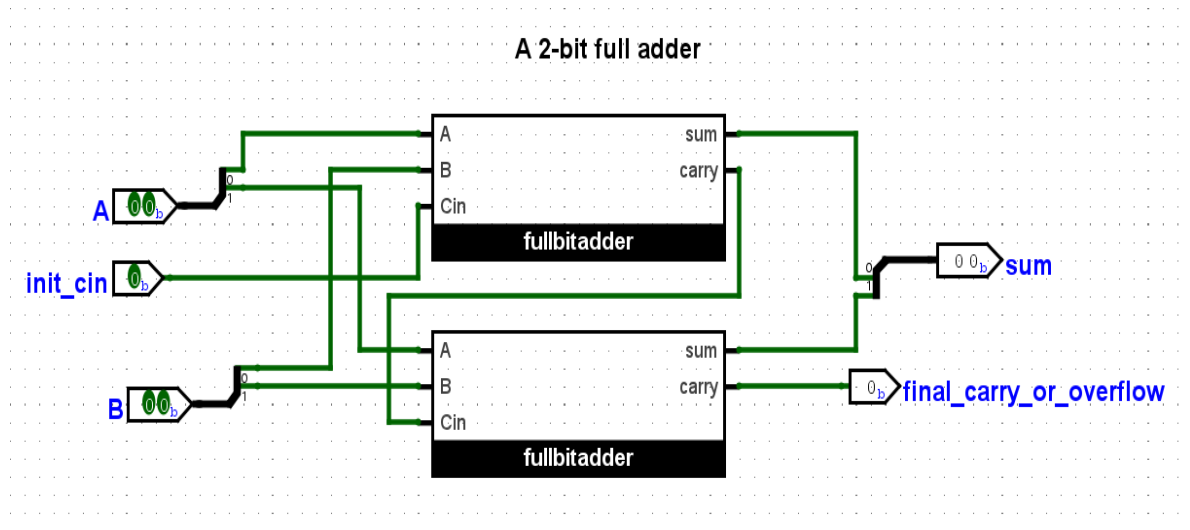


Figure 3: 2-bit full adder

- Compute the additions for the following as 2-bit additions. Indicate their Sum and Carry (overflow).
 1. $00 + 11$
 2. $10 + 01$
 3. $11 + 11$
 4. $01 + 11$
- Test and simulate your 2-bit half-adder for the above computations and ensure that your circuit's output matches your computed result. Attach screenshots of your simulations to your submission. You can name them as *2bitfulladder_sim1*,...*bitfulladder_sim4*.

Part 3: Extending the 2-bit Adder to Perform Subtraction

The main challenge in this part is to extend the 2-bit adder from the previous section to perform subtraction. Assume that your adder has two 2-bit inputs: A and B. To perform subtraction, find the negative equivalent of value (input) B, and add to A.

$$\text{difference} = A - B$$

$$= A + (-1) B$$

We know that the 2's Complement of a binary number can be computed by inverting every bit in the number and adding 1. Therefore, to find the negative equivalent of B, invert the value of B using an inverter and add 1.

An Inverter vs a Controlled Inverter

An inverter is a gate that has 1 input and one output, and the output is the inverse of the input. If the input is 1 then the output is 0; if the input is 0 the output is 1.

A controlled inverter has two inputs: the data input and the control input. The inverter functions as an inverter when the control value is 1. If the control value is 0, it does not invert the value at the data input; rather, it just passes the data as it is to the output. In our case, we will use a controlled inverter so that we can tell when our circuit should perform an addition or a subtraction.

An **XOR** gate can be used as a controlled inverter. Remember, the XOR Gate outputs 0 when both inputs are the same, and 1 otherwise.

For example: If our B-input bit is 1 and our control is on (set to 1), the output of the XOR gate results to 0, which inverts the B-input bit of 1. Now, if we have a B-input bit of 0 and our control is on as usual (set to 1), the output of the XOR Gate results in 1, thereby inverting our input bit of 0. By this, we can confirm that the XOR Gate can be used as a controlled inverter.

- Add a new circuit and name it as addsub2bit.
- Highlight every component in your 2-bit adder circuit and press Ctrl/Cmd + C to copy. Paste in your new circuit worksheet.
- Edit your new circuit and include XOR gates and a control pin to change the values of B into 2's complement and fulfill the subtraction operation as shown in the image below:

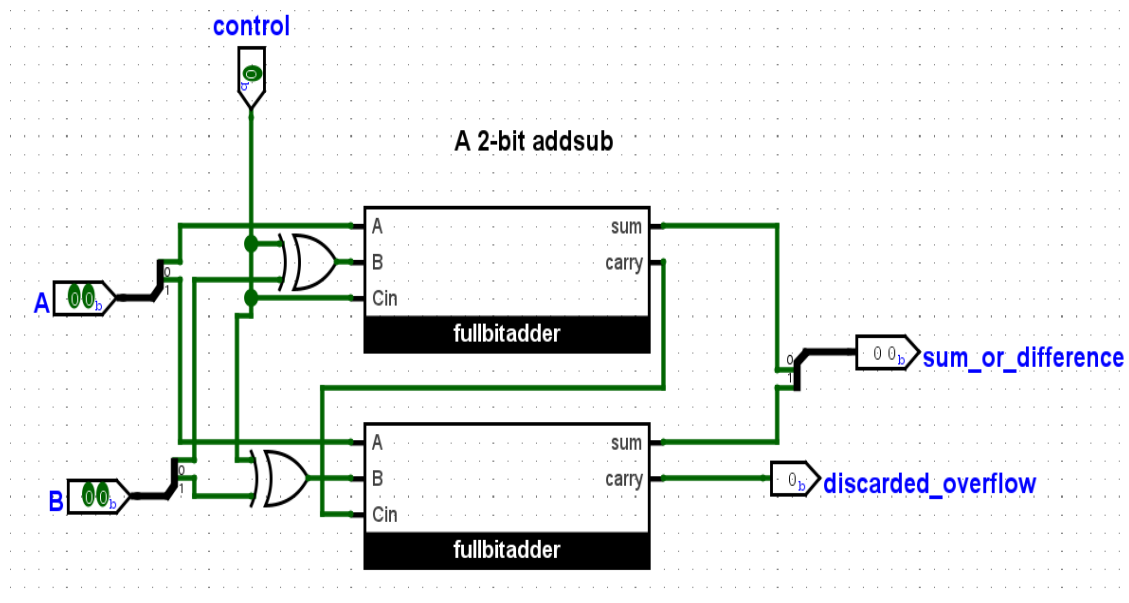


Figure 4: 2-bit addsub

NB: During subtraction, if B is greater than A and we are expecting a negative result, then the output of the subtraction will be in 2s complement form (denoting a negative outcome).

- Compute the arithmetic for the following as 2-bit subtractions using the 2's complement approach. Indicate their Sum and Carry (overflow).
 1. 00 - 11
 2. 10 - 01
 3. 11 - 11
 4. 01 - 11
- Test and simulate your 2-bit half-adder for the above computations and ensure that your circuit's output matches your computed result. Attach screenshots of your simulations to your submission. You can name them as *2bitaddsub_sim1*,...*2bitaddsub_sim4*.

Part 4: Designing an 8-bit Full Adder

Aim: Design an 8-bit full adder that performs addition.

In this practical, you will build an 8-bit full adder (a module that will accept two 8-bit numbers and add them to generate an 8-bit sum as output) by combining 8 single full adders.

- Add a new circuit and name as ***fulladder8bit***. Add 8 objects of the *fullbitadder* module to the worksheet. Interconnect these components to create an 8-bit adder.
- Compute the additions for the following as 8-bit additions. Indicate their Sum and Carry.
 1. $1010110 + 110100$
 2. $11001011 + 1011010$
 3. $110001 + 11100100$
 4. $1010001 + 11001$
- Test and simulate your 8-bit adder for the above computations and ensure that your circuit's output matches your computed result. Attach screenshots of your simulations to your submission. You can name them as ***8bitfulladder_sim1,...8bitfulladder_sim4***

Part 5: Extending your 8-bit Full Adder to Perform Subtraction

- Add a new circuit and name it as ***addsub8bit***. Create a copy of your 8-bit full adder and include the controlled inverter to extend the circuit to perform subtraction.
- Compute the arithmetic for the following as 8-bit subtractions using the 2's complement approach. Indicate their Sum and Carry.
 1. $FF - E6$
 2. $A5 - 6F$
 3. $F2 - D7$
 4. $110001 - 11100100$
 5. $1010001 - 11001$
- Test and simulate your 8-bit adder for the above computations and ensure that your circuit's output matches your computed result. Attach screenshots of your simulations to your submission. You can name them as ***8bitaddsub_sim1,...8bitaddsub_sim4***.

Part 6: Reflection Questions

1. How does the digital logic of a full adder circuit differ from that of a half adder?
2. What role does the XOR gate play in the implementation of a full adder circuit?
3. How does the carry-out bit in a full adder circuit influence subsequent addition operations?
4. Can a full adder circuit be used to perform subtraction? If so, how?
5. What is the significance of the XOR and AND gates in a subtractor circuit?

Submission:

Please submit the following in a zip file named as ID_Lab3

- Your Logisim circuit file
- PDF of written responses